

INSTITUTO TECNOLÓGICO SUDAMERICANO

Autor:

Jairo Arevalo

Asignatura:

Herramientas informáticas para el despliegue de diagramas

Actividad:

Realizar un backen en la cual se utilize recursos temporales

Carrera:

Desarrollo de Software

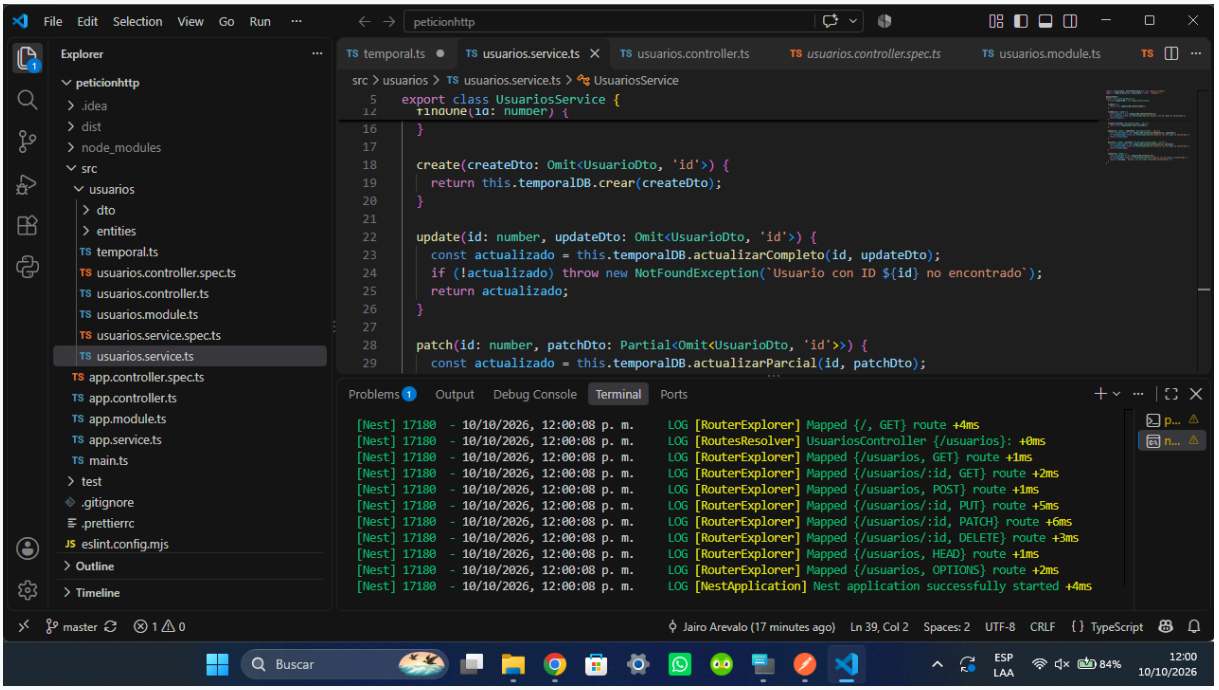
Clase:

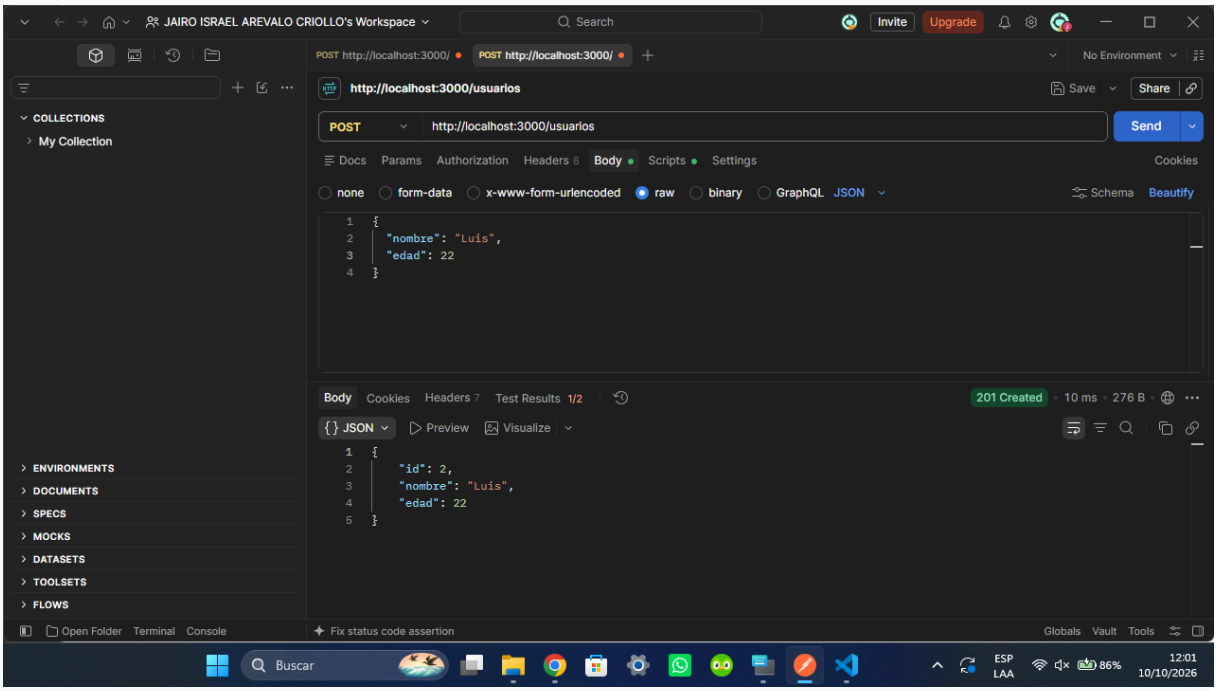
3er Ciclo

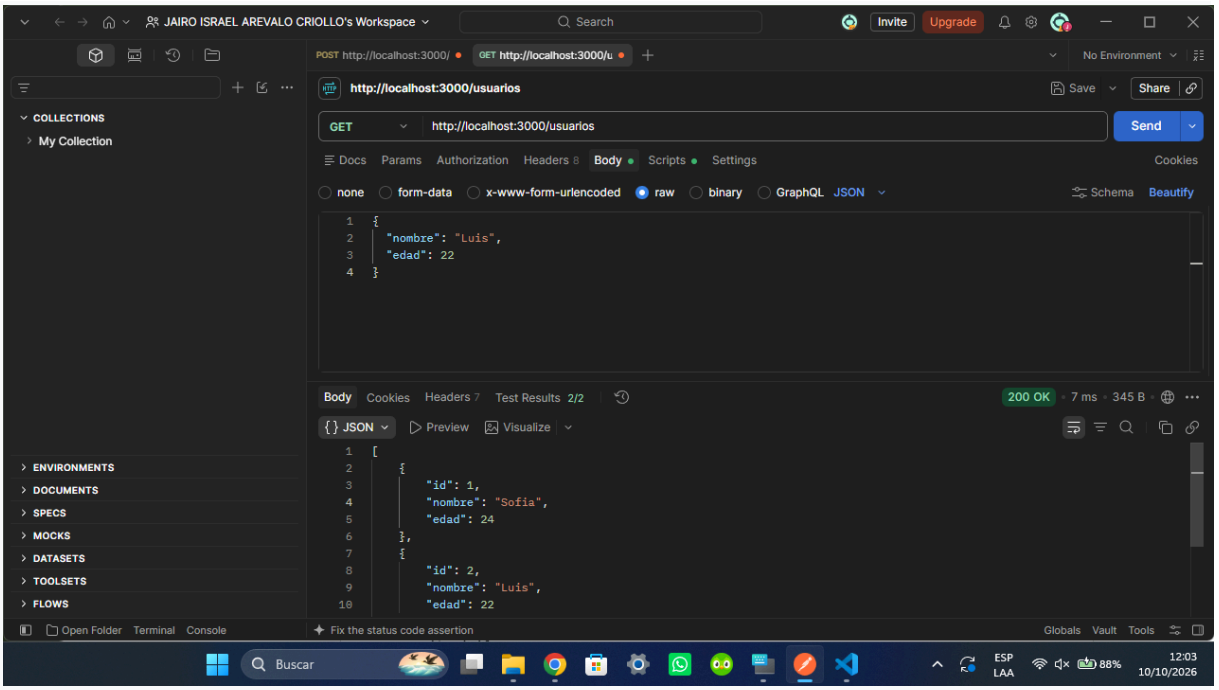
Fecha:

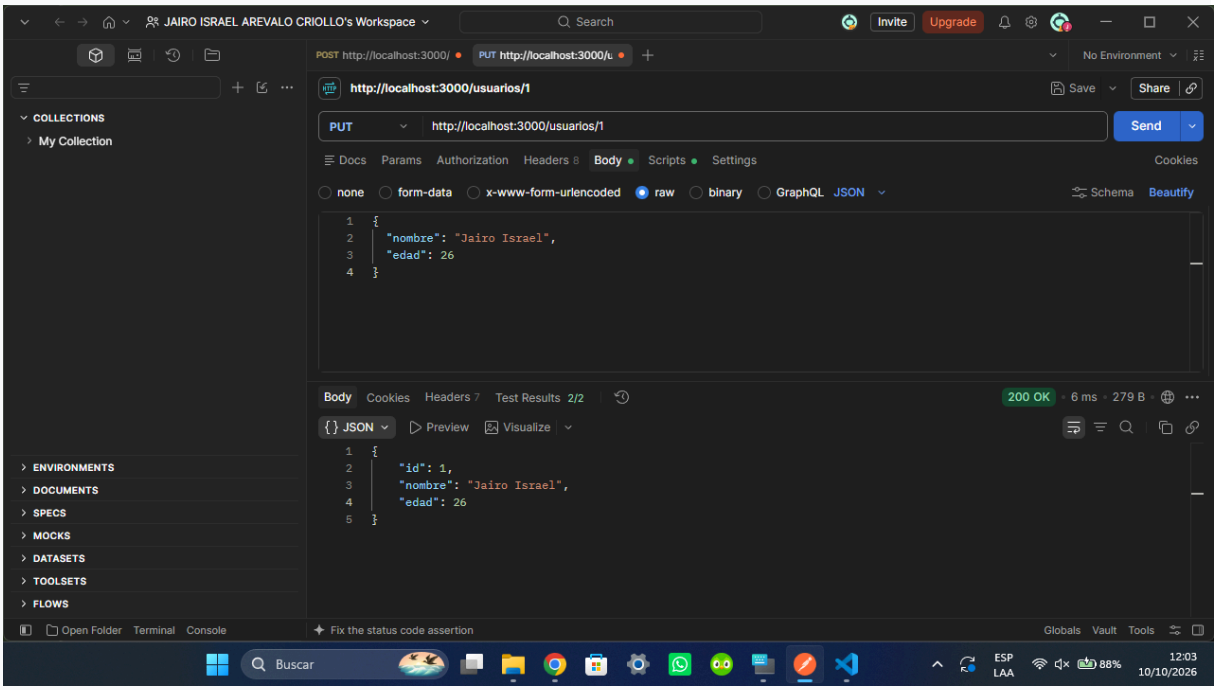
10 de octubre del 2026

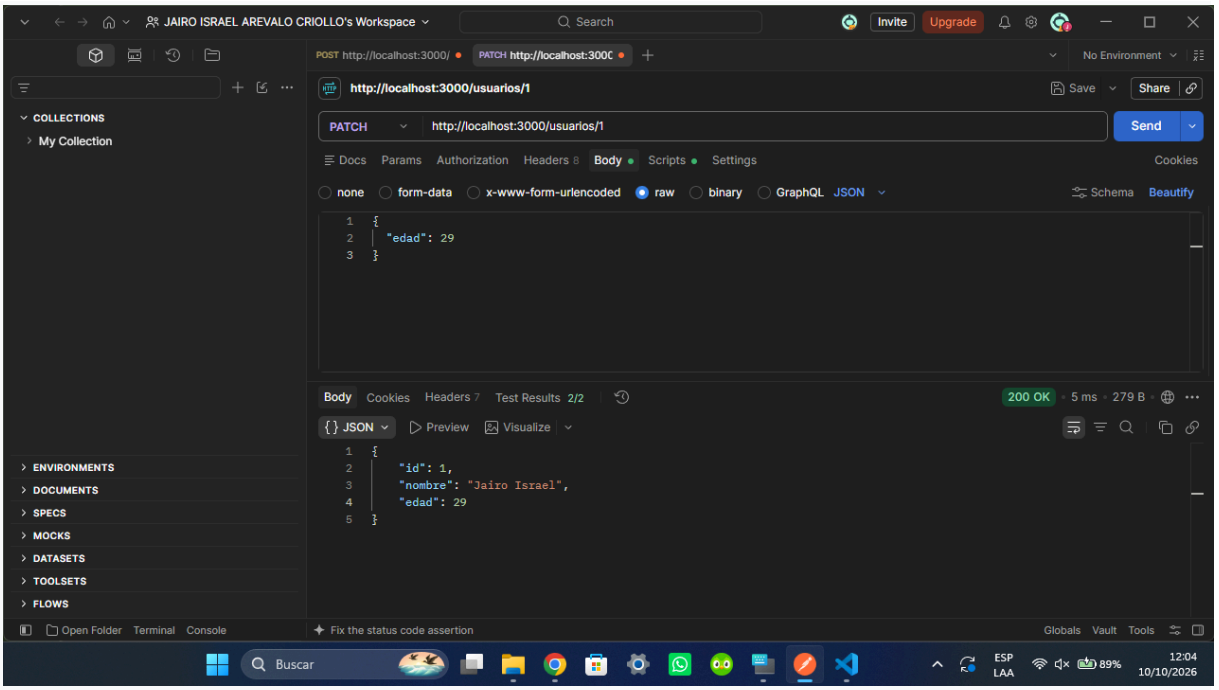
Enlace repositorio: https://github.com/jiarevalo-hub/peticionhttp

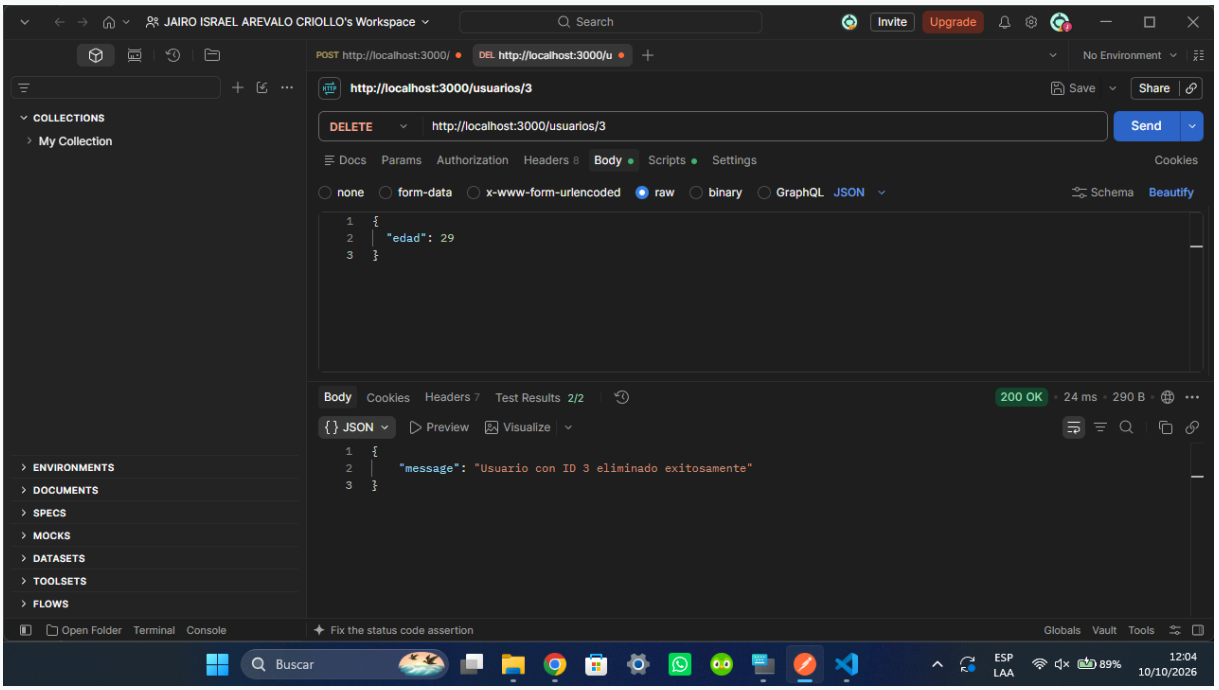


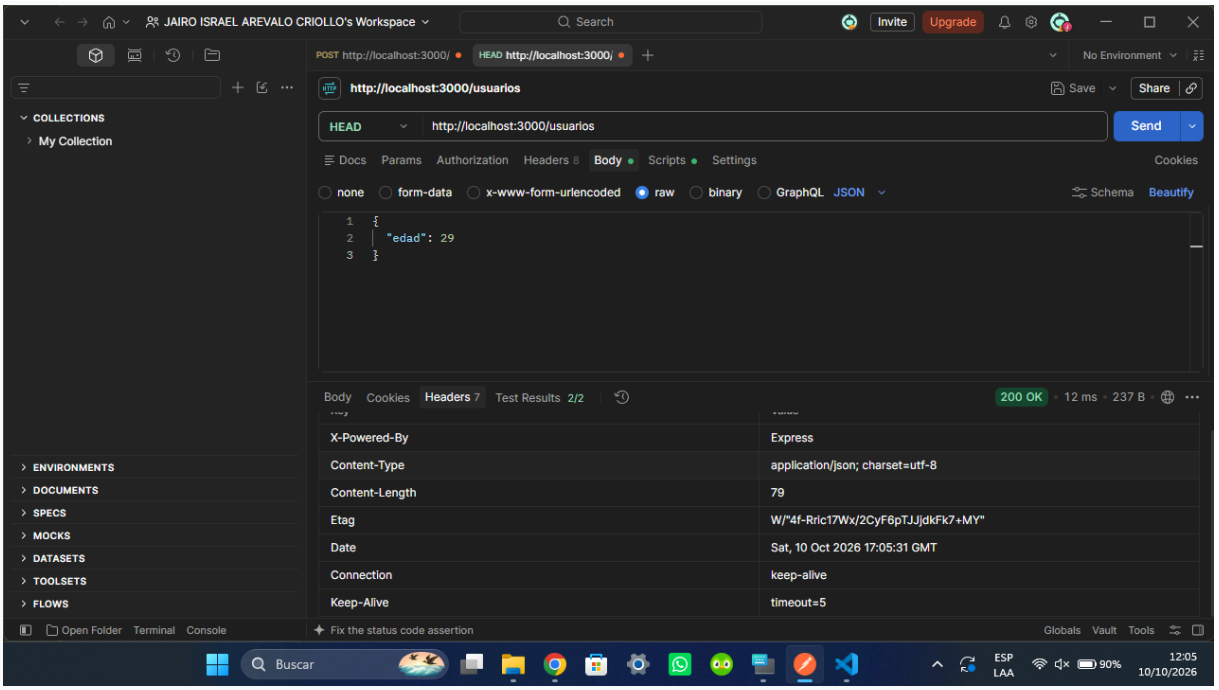


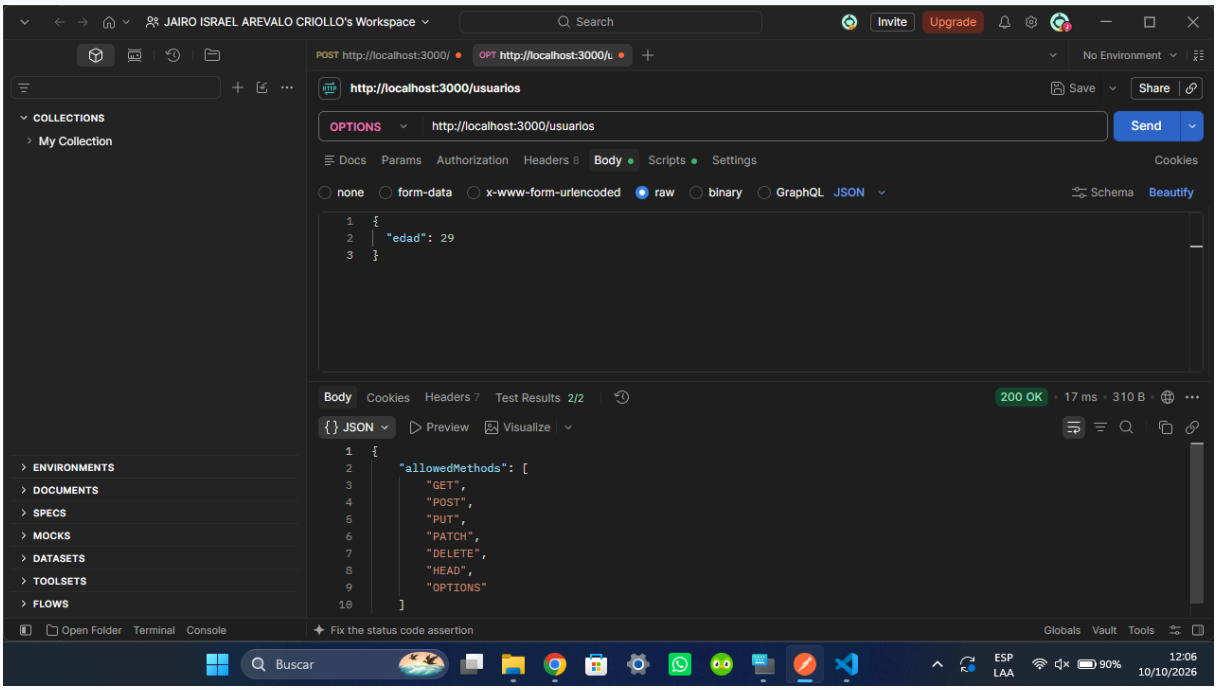

JAIRO ISRAEL AREVALO CRIOLLO's Workspace
Search
Invite
Upgrade
POST http://localhost:3000/
GET http://localhost:3000/u
No Environment
http://localhost:3000/usuarios
Save
Share
COLLECTIONS
My Collection
GET
http://localhost:3000/usuarios
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"nombre": "Luis",
"edad": 22
}
Body
Cookies
Headers 7
Test Results 2/2
200 OK
7 ms
345 B
{ } JSON
Preview
Visualize
[
{
"id": 1,
"nombre": "Sofia",
"edad": 24
},
{
"id": 2,
"nombre": "Luis",
"edad": 22
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
TOOLSETS
FLOWS
Open Folder
Terminal
Console
Fix the status code assertion
Globals
Vault
Tools
Buscar
ESP
LAA
88%
12:03
10/10/2026


JAIRO ISRAEL AREVALO CRIOLLO's Workspace
Search
Invite
Upgrade
POST http://localhost:3000/
PUT http://localhost:3000/u
No Environment
http://localhost:3000/usuarios/1
Save
Share
COLLECTIONS
My Collection
PUT
http://localhost:3000/usuarios/1
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"nombre": "Jairo Israel",
"edad": 26
}
Body
Cookies
Headers 7
Test Results 2/2
200 OK
6 ms
279 B
{ } JSON
Preview
Visualize
{
"id": 1,
"nombre": "Jairo Israel",
"edad": 26
}
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
TOOLSETS
FLOWS
Open Folder
Terminal
Console
Fix the status code assertion
Globals
Vault
Tools
Buscar
ESP
LAA
88%
12:03
10/10/2026


JAIRO ISRAEL AREVALO CRIOLLO's Workspace
Search
Invite
Upgrade
No Environment
http://localhost:3000/usuarios/1
Save
Share
PATCH
http://localhost:3000/usuarios/1
Send
Docs Params Authorization Headers 8 Body Scripts Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"edad": 29
}
Body Cookies Headers 7 Test Results 2/2
200 OK 5 ms 279 B
{ } JSON Preview Visualize
{
"id": 1,
"nombre": "Jairo Israel",
"edad": 29
}
COLLECTIONS
My Collection
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
TOOLSETS
FLOWS
Open Folder Terminal Console
Fix the status code assertion
Globals Vault Tools
Buscar
ESP LAA
89%
12:04 10/10/2026


JAIRO ISRAEL AREVALO CRIOLLO's Workspace
Search
Invite
Upgrade
No Environment
http://localhost:3000/usuarios/3
Save
Share
DELETE
http://localhost:3000/usuarios/3
Send
Docs Params Authorization Headers 8 Body Scripts Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"edad": 29
}
Body Cookies Headers 7 Test Results 2/2
200 OK 24 ms 290 B
{ } JSON Preview Visualize
{
"message": "Usuario con ID 3 eliminado exitosamente"
}
COLLECTIONS
My Collection
ENVIRONMENTS
DOCUMENTS
SPECS
MOCKS
DATASETS
TOOLSETS
FLOWS
Open Folder Terminal Console
Fix the status code assertion
Globals Vault Tools
Buscar
ESP LAA
89%
12:04 10/10/2026





Uso de IA

Inteligencia Artificial Utilizada: Gemini

1. Establecimiento de Rol

- **Prompt del usuario:** "Actúa como un programador de backend profesional"
- **Objetivo:** Definir el perfil y el contexto técnico de la conversación.
- **Información y palabras clave obtenidas:** Activación del rol de programador de backend, enfocado en arquitectura, diseño de APIs, bases de datos y microservicios.

- **Cómo llevó a la siguiente consulta:** Al fijar este perfil técnico, propiciaste plantear de inmediato un caso práctico de desarrollo en NestJS.

2. Diseño del Backend en NestJS y Archivo temporal.ts

- **Prompt del usuario:** "quiero hacer un backend en nest para realizar pruebas en POSTMAN (GET, POST, PUT, PATCH, DELETE, HEAD Y OPTIONS)... genere un nuevo recurso llamado usuarios, dentro del recurso usuarios crea un archivo llamado temporal.ts..."
- **Objetivo:** Construir la estructura inicial con un arreglo en memoria para manejar datos de usuarios y cubrir los 7 métodos HTTP solicitados.
- **Información y palabras clave obtenidas:** Estructura de NestJS, recurso usuarios, archivo temporal.ts con clase, constructor y arreglo inicial con id, nombre y edad, además de controladores y servicios asociados.
- **Cómo llevó a la siguiente consulta:** La entrega de este bloque de código motivó la necesidad de entender cómo ejecutar y verificar cada verbo HTTP en Postman.

3. Guía de Pruebas en Postman

- **Prompt del usuario:** "ahora quiero hacer las pruebas con todos los metodos de POSTMAN"
- **Objetivo:** Proveer instrucciones paso a paso sobre cómo configurar y ejecutar los 7 métodos HTTP en la herramienta.
- **Información y palabras clave obtenidas:** Endpoints, verbos (GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS), cuerpos en formato JSON y respuestas esperadas por el servidor.
- **Cómo llevó a la siguiente consulta:** Una vez comprendido el flujo de pruebas, el usuario solicitó una revisión exhaustiva del código fuente que sustentaba toda la lógica.